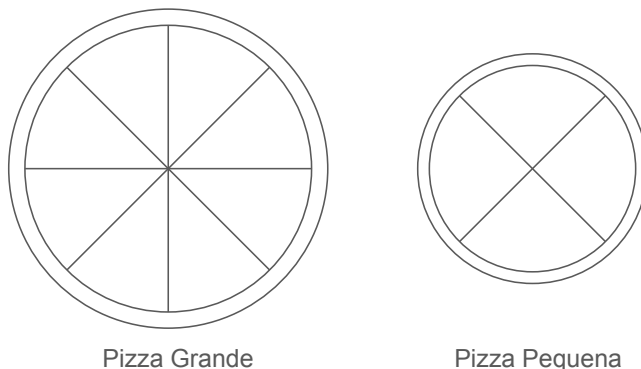


# Pizzaria

Nome do arquivo: `pizzaria.c`, `pizzaria.cpp`, `pizzaria.pas`, `pizzaria.java`, `pizzaria.js` ou `pizzaria.py`

A mãe de Larissa decidiu comemorar o aniversário da filha em uma pizzaria. Nessa pizzaria, existem pizzas de dois tamanhos: uma pizza grande possui 8 fatias, enquanto que uma pizza pequena possui 4 fatias.



A pizzaria informou que os pedidos devem ser feitos com antecedência. Por isso, a mãe de Larissa já fez um pedido indicando quantas pizzas de cada tamanho ela vai comprar.

Larissa está empolgada em convidar seus amigos para a festa. Porém, como ela tem muitos amigos, é possível que o pedido feito não possua fatias de pizza suficientes para servir ela, sua mãe e todos os seus amigos. Assim, Larissa gostaria de saber, no máximo, quantos amigos ela pode convidar de modo que todos os convidados (ela, sua mãe e seus amigos) recebam uma fatia de pizza.

Por exemplo, suponha que a mãe de Larissa pediu 2 pizzas grandes e 3 pizzas pequenas. No total, haverão  $2 \times 8 + 3 \times 4 = 28$  fatias de pizza. Após reservar uma fatia para Larissa e uma para sua mãe, sobram  $28 - 2 = 26$  fatias de pizza. Assim, Larissa poderá convidar 26 amigos para sua festa.

Sua tarefa é: dadas a quantidade de pizzas grandes e a quantidade de pizzas pequenas que a mãe de Larissa pediu, calcule o número máximo de **amigos** que Larissa pode convidar para a festa, ou seja, o número de fatias que sobram após reservar uma fatia para Larissa e uma para sua mãe.

## Entrada

A entrada possui duas linhas. A primeira linha contém um inteiro  $G$ , indicando a quantidade de pizzas grandes que a mãe de Larissa pediu. A segunda linha contém um inteiro  $P$ , indicando a quantidade de pizzas pequenas que ela pediu.

## Saída

Seu programa deverá imprimir uma única linha contendo um único inteiro: o número máximo de amigos que Larissa pode convidar para a festa, excluindo ela mesma e sua mãe.

## Restrições

É garantido que todo caso de teste satisfaz as restrições abaixo.

- $1 \leq G \leq 10$
- $1 \leq P \leq 10$

**Informações sobre a pontuação**

A tarefa vale 100 pontos.

**Exemplos**

| Exemplo de entrada 1 | Exemplo de saída 1 |
|----------------------|--------------------|
| 2<br>3               | 26                 |

*Explicação do exemplo 1:* O inteiro 2 na primeira linha indica que foram compradas duas pizzas grandes, enquanto que o inteiro 3 na segunda linha indica que foram compradas três pizzas pequenas. Logo, este é o exemplo mostrado no enunciado. Por isso, imprimimos a resposta correta,  $2 \times 8 + 3 \times 4 - 2 = 26$ . Observe que a saída correta é **apenas** o inteiro 26, sem nenhum outro texto ou mensagem para o usuário.

| Exemplo de entrada 2 | Exemplo de saída 2 |
|----------------------|--------------------|
| 7<br>4               | 70                 |

*Explicação do exemplo 2:* Neste caso, haverão  $7 \times 8 + 4 \times 4 = 72$  fatias de pizza. Reservando duas fatias para Larissa e sua mãe, sobram  $72 - 2 = 70$  fatias para os amigos de Larissa.

| Exemplo de entrada 3 | Exemplo de saída 3 |
|----------------------|--------------------|
| 1<br>2               | 14                 |

*Explicação do exemplo 3:* Neste caso, a resposta é  $1 \times 8 + 2 \times 4 - 2 = 14$ .

# Café com Leite

*Nome do arquivo:* `leite.c`, `leite.cpp`, `leite.pas`, `leite.java`, `leite.js` ou `leite.py`

Felipe trabalha em uma cafeteria especializada em café com leite. O chefe dele criou uma promoção na qual os clientes recebem um desconto caso tragam suas próprias xícaras, evitando o uso de materiais descartáveis. A promoção se tornou muito popular, o que é ótimo para o meio ambiente mas dificultou o trabalho de Felipe, pois cada cliente possui uma xícara de um tamanho diferente. Além disso, cada cliente prefere quantidades diferentes de leite na bebida.

Ao fazer um pedido, o cliente indica para Felipe dois números: o volume mínimo  $A$  (em mililitros) e o volume máximo  $B$  (em mililitros) de leite que ele deseja em sua bebida. O cliente indica também a capacidade  $C$  (também em mililitros) de sua xícara.

Para preparar o pedido, Felipe insere a xícara na máquina de espresso, que prepara  $D$  mililitros de café. Ele usa a máquina somente uma vez, isto é, o volume de café na xícara sempre será exatamente  $D$ . Depois de remover a xícara da máquina, Felipe adiciona leite de modo a enchê-la completamente, ou seja, o volume total de café com leite é exatamente  $C$ .

Felipe gostaria de saber se o volume de leite na xícara está dentro das preferências do cliente. Por exemplo, suponha que a máquina prepare  $D = 30$  ml de café e considere dois clientes:

- O cliente 1 possui uma xícara com capacidade  $C = 170$  ml e deseja que sua bebida possua entre  $A = 130$  ml e  $B = 150$  ml de leite. Neste caso, a bebida preparada por Felipe possui 30 ml de café e  $170 - 30 = 140$  ml de leite. O volume de leite está entre 130 e 150 ml, ou seja, dentro das preferências do cliente.
- O cliente 2 possui uma xícara com capacidade  $C = 240$  ml e deseja que sua bebida possua entre  $A = 220$  ml e  $B = 230$  ml de leite. Neste caso, a bebida preparada por Felipe possui 30 ml de café e  $240 - 30 = 210$  ml de leite. Portanto, o volume de leite está abaixo do volume mínimo especificado pelo cliente.

Escreva um programa para ajudar Felipe: dados os volumes  $A$ ,  $B$  e  $C$  especificados por um cliente e o volume  $D$  de café preparado pela máquina, determine se o volume de leite na bebida atenderá às preferências do cliente.

## Entrada

A entrada possui quatro linhas, cada uma contendo um único inteiro:

- a primeira linha contém o volume mínimo  $A$  de leite (em ml) que o cliente deseja;
- a segunda linha contém o volume máximo  $B$  de leite (em ml) que o cliente deseja;
- a terceira linha contém a capacidade  $C$  (em ml) da xícara;
- a quarta linha contém o volume  $D$  (em ml) de café preparado pela máquina.

## Saída

Seu programa deverá imprimir uma única linha contendo um único caractere: caso Felipe consiga satisfazer as preferências do cliente, imprima o caractere **S** (a letra S maiúscula). Caso contrário, imprima o caractere **N** (a letra N maiúscula).

## Restrições

É garantido que todo caso de teste satisfaz as restrições abaixo.

- $100 \leq C \leq 500$
- $0 \leq A \leq B < C$
- $10 \leq D \leq 100$

## Informações sobre a pontuação

A tarefa vale 100 pontos.

## Exemplos

| Exemplo de entrada 1    | Exemplo de saída 1 |
|-------------------------|--------------------|
| 130<br>150<br>170<br>30 | S                  |

*Explicação do exemplo 1:* Este exemplo corresponde ao cliente 1 do enunciado.

| Exemplo de entrada 2    | Exemplo de saída 2 |
|-------------------------|--------------------|
| 220<br>230<br>240<br>30 | N                  |

*Explicação do exemplo 2:* Este exemplo corresponde ao cliente 2 do enunciado.

| Exemplo de entrada 3  | Exemplo de saída 3 |
|-----------------------|--------------------|
| 0<br>200<br>300<br>45 | N                  |

*Explicação do exemplo 3:* Neste caso, o volume de leite na bebida é  $300 - 45 = 255$  ml, o que excede o limite máximo de 200 ml especificado pelo cliente.

| Exemplo de entrada 4    | Exemplo de saída 4 |
|-------------------------|--------------------|
| 120<br>120<br>200<br>80 | S                  |

# Dieta

*Nome do arquivo:* `dieta.c`, `dieta.cpp`, `dieta.pas`, `dieta.java`, `dieta.js` ou `dieta.py`

O gato Garfield comeu lasanhas demais nos últimos dias, o que está afetando seu metabolismo. Por isso, seu dono John decidiu colocá-lo em uma dieta muito rígida.

Seguindo as instruções do método SBC (*Seleção Benéfica de Calorias*), John definiu um limite  $M$  de calorias que o gato poderia consumir diariamente. Para não perder as contas de quantas calorias Garfield já consumiu no dia, John observa o rótulo das lasanhas e anota em uma lista as quantidades em gramas de proteínas, gorduras e carboidratos presentes em cada uma das  $N$  refeições do gato.

Para calcular quantas calorias Garfield já consumiu, John utiliza a seguinte conversão:

- 1 grama de proteína tem 4 calorias.
- 1 grama de gordura tem 9 calorias.
- 1 grama de carboidrato tem 4 calorias.

John é um humano e consegue calcular isso facilmente. Porém, Garfield é apenas um gato que gosta de comer. Portanto, dada a lista de refeições que Garfield já fez, ajude o gato a saber qual o máximo de calorias que ele ainda pode consumir, sem exceder o limite  $M$  determinado.

## Entrada

A primeira linha da entrada contém dois inteiros  $N$  e  $M$ : a quantidade de refeições na lista de John e o limite de calorias, respectivamente.

Cada uma das  $N$  linhas seguintes contém três inteiros,  $P$ ,  $G$  e  $C$ : as quantidades (em gramas) de proteínas, gorduras e carboidratos, respectivamente, de uma refeição na lista de John.

## Saída

Seu programa deverá imprimir uma única linha contendo um único inteiro: a quantidade máxima de calorias que Garfield ainda pode consumir sem exceder o limite  $M$ .

## Restrições

É garantido que todo caso de teste satisfaz as restrições abaixo.

- $1 \leq N \leq 30$
- $1 \leq M \leq 300\,000$
- $0 \leq P, G, C \leq 500$
- O total de calorias nas refeições na lista de John não excede o limite  $M$

## Informações sobre a pontuação

A tarefa vale 100 pontos. Estes pontos estão distribuídos em subtarefas, cada uma com suas **restrições adicionais** às definidas acima.

- **Subtarefa 1 (0 pontos):** Esta subtarefa é composta apenas pelos exemplos mostrados abaixo. Ela não vale pontos, serve apenas para que você verifique se o seu programa imprime o resultado correto para os exemplos.
- **Subtarefa 2 (30 pontos):**  $N = 1$ .
- **Subtarefa 3 (70 pontos):** Sem restrições adicionais.

**Exemplos**

| Exemplo de entrada 1                       | Exemplo de saída 1 |
|--------------------------------------------|--------------------|
| 3 2000<br>65 15 20<br>40 20 25<br>50 10 35 | 655                |

*Explicação do exemplo 1:* Aqui temos 3 refeições e a meta diária são 2000 calorias.

- A primeira refeição possui 65g de proteínas, 15g de gorduras e 20g de carboidratos, totalizando  $65 \times 4 + 15 \times 9 + 20 \times 4 = 475$  calorias.
- A segunda refeição possui 40g de proteínas, 20g de gorduras e 25g de carboidratos, totalizando  $40 \times 4 + 20 \times 9 + 25 \times 4 = 440$  calorias.
- A terceira refeição possui 50g de proteínas, 10g de gorduras e 35g de carboidratos, totalizando  $50 \times 4 + 10 \times 9 + 35 \times 4 = 430$  calorias.

No total, foram consumidas  $475 + 440 + 430 = 1345$  calorias. Garfield ainda pode consumir  $2000 - 1345 = 655$  calorias.

| Exemplo de entrada 2 | Exemplo de saída 2 |
|----------------------|--------------------|
| 1 3700<br>50 300 200 | 0                  |

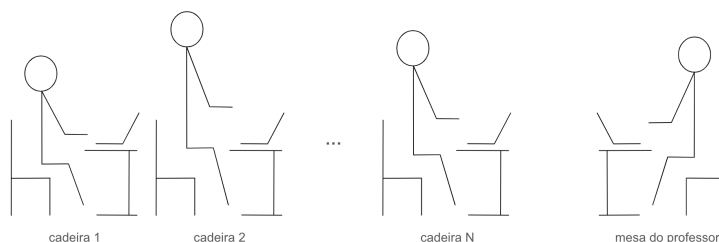
*Explicação do exemplo 2:* Garfield fez apenas uma refeição com  $50 \times 4 + 300 \times 9 + 200 \times 4 = 3700$  calorias. Como 3700 é o limite de calorias estabelecido por John, Garfield não pode comer mais nenhuma caloria. (*Este exemplo satisfaz as restrições da subtarefa 2.*)

# Fila

Nome do arquivo: `fila.c`, `fila.cpp`, `fila.pas`, `fila.java`, `fila.js` ou `fila.py`

Após um longo semestre de aulas de informática, chegou a hora do professor Ricardo avaliar o conhecimento de seus alunos com uma prova surpresa. No entanto, ele conhece seus alunos e sabe que nem todos estão com a matéria em dia. Por isso, ele suspeita que alguns deles tentarão colar na prova, algo que ele não pode permitir!

O professor é muito bom em identificar trapagens e consegue reconhecê-las com apenas um olhar. No entanto, sua sala de aula é um tanto incomum. Nessa sala, as cadeiras dos  $N$  alunos estão dispostas em uma única fila, de forma que a cadeira 1 fica no fundo da sala e a cadeira  $N$  fica logo em frente à mesa de Ricardo, de onde ele vigia os alunos.



Os alunos possuem tamanhos variados, sendo que o aluno sentado na cadeira  $i$  tem  $a_i$  centímetros de altura. Por causa da organização estranha das cadeiras, essas alturas podem atrapalhar a visão do professor, de modo que Ricardo não consegue enxergar um aluno caso exista outro aluno com altura maior ou igual à dele que está mais perto da mesa do professor. Formalmente, Ricardo **não** consegue enxergar o aluno sentado na cadeira  $i$  se existe algum aluno em uma cadeira  $j$  tal que  $j > i$  e  $a_j \geq a_i$ . Observe que o aluno na cadeira  $N$  sempre pode ser observado, pois está logo em frente ao professor.

Ricardo consegue monitorar os alunos que ele enxerga, mas está preocupado com possíveis colas entre os demais. Dadas as alturas dos alunos, ajude-o a saber qual o máximo de alunos que poderão colar na prova sem que ele perceba, isto é, a quantidade de alunos que ele não consegue enxergar.

## Entrada

A primeira linha da entrada contém um único inteiro positivo  $N$ , a quantidade de alunos. A segunda linha da entrada contém  $N$  inteiros positivos  $a_1, a_2, \dots, a_N$ , as alturas dos alunos na ordem em que eles estão dispostos na sala.

## Saída

A saída deve conter um único inteiro, a quantidade de alunos que Ricardo **não** consegue enxergar.

## Restrições

É garantido que todo caso de teste satisfaz as restrições abaixo.

- $1 \leq N \leq 100\,000$
- $1 \leq a_i \leq 1\,000\,000\,000$  para todo  $1 \leq i \leq N$

## Informações sobre a pontuação

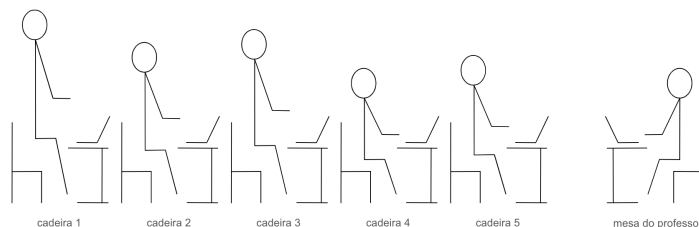
A tarefa vale 100 pontos. Estes pontos estão distribuídos em subtarefas, cada uma com suas **restrições adicionais** às definidas acima.

- **Subtarefa 1 (0 pontos):** Esta subtarefa é composta apenas pelos exemplos mostrados abaixo. Ela não vale pontos, serve apenas para que você verifique se o seu programa imprime o resultado correto para os exemplos.
- **Subtarefa 2 (60 pontos):**  $N \leq 1\,000$ .
- **Subtarefa 3 (40 pontos):** Sem restrições adicionais.

## Exemplos

| Exemplo de entrada 1     | Exemplo de saída 1 |
|--------------------------|--------------------|
| 5<br>200 180 190 140 160 | 2                  |

*Explicação do exemplo 1:* Note que o professor Ricardo está localizado à direita dos alunos. O segundo aluno não pode ser visto, porque o terceiro é mais alto que ele. Da mesma forma, o quarto aluno não pode ser visto, porque é mais baixo que o quinto. Conforme ilustrado pela figura a seguir:



| Exemplo de entrada 2     | Exemplo de saída 2 |
|--------------------------|--------------------|
| 5<br>180 180 170 150 190 | 4                  |

*Explicação do exemplo 2:* Como o maior aluno da sala está logo em frente à Ricardo, os quatro outros alunos atrás dele não podem ser vistos pelo professor. Logo, a resposta é 4.

| Exemplo de entrada 3 | Exemplo de saída 3 |
|----------------------|--------------------|
| 2<br>150 150         | 1                  |

*Explicação do exemplo 3:* Como o segundo aluno tem a mesma altura que o primeiro, este não pode ser visto pelo professor. Logo, a resposta é 1.



# Festa Junina

*Nome do arquivo:* `festa.c`, `festa.cpp`, `festa.pas`, `festa.java`, `festa.js` ou `festa.py`

A escola de Luísa está se preparando para a festa junina deste ano. Para este grande evento, a cozinha da escola precisa de ingredientes para preparar pratos típicos como canjica e pamonha. Além disso, os alunos participarão de uma dança de quadrilha, para a qual eles precisam de roupas tradicionais como camisas xadrez e chapéus de palha.

A professora de Luísa pediu ajuda a ela para comprar ingredientes e roupas para a festa. Luísa irá comprar as roupas na lojinha do bairro e os ingredientes no supermercado. A escola, o supermercado e a lojinha estão localizados na mesma rua reta. A posição de cada um destes três prédios pode ser representada por um inteiro indicando a distância (em metros) do prédio ao início da rua. A distância percorrida para ir de um prédio a outro é dada pela diferença entre as posições deles.

Luísa atualmente está na escola e precisa visitar o supermercado e a lojinha, em qualquer ordem. Em seguida, ela deve voltar com as compras para a escola. Ajude Luísa a descobrir qual a distância que ela precisa percorrer, no mínimo, para fazer todas as compras e voltar para a escola.

Por exemplo, se as posições da escola, supermercado e lojinha são 10, 5 e 13, respectivamente, Luísa pode ir primeiro ao supermercado, percorrendo  $10 - 5 = 5$  metros, então ir do supermercado à lojinha, percorrendo  $13 - 5 = 8$  metros, e então voltar para a escola, percorrendo  $13 - 10 = 3$  metros. No total, a distância que ela vai percorrer será  $5 + 8 + 3 = 16$  metros. É possível verificar que, caso ela escolha visitar primeiro a lojinha e depois o supermercado, ela também percorrerá 16 metros no total.

## Entrada

A entrada possui três linhas, cada uma contendo um único inteiro:

- a primeira linha contém o inteiro  $E$ , a posição da escola;
- a segunda linha contém o inteiro  $S$ , a posição do supermercado;
- a terceira linha contém o inteiro  $L$ , a posição da lojinha.

## Saída

Seu programa deverá imprimir uma única linha contendo um único inteiro, a distância total em metros que Luísa precisa percorrer.

## Restrições

É garantido que todo caso de teste satisfaz as restrições abaixo.

- $0 \leq E, S, L \leq 1\,000$
- As posições dos três locais são distintas.

## Informações sobre a pontuação

A tarefa vale 100 pontos.

**Exemplos**

| Exemplo de entrada 1 | Exemplo de saída 1 |
|----------------------|--------------------|
| 10<br>5<br>13        | 16                 |

*Explicação do exemplo 1:* Este é o exemplo mostrado no enunciado: a primeira linha indica que a posição da escola é  $E = 10$ , a segunda linha indica que a posição do supermercado é  $S = 5$  e a terceira linha indica que a posição da lojinha é  $L = 13$ . A resposta é 16 metros, portanto, a saída contém o inteiro 16. Observe que a saída correta é **apenas** o inteiro 16, sem nenhum outro texto ou mensagem para o usuário.

| Exemplo de entrada 2 | Exemplo de saída 2 |
|----------------------|--------------------|
| 4<br>25<br>17        | 42                 |

*Explicação do exemplo 2:* Neste caso, a escola está à esquerda dos outros dois prédios. Uma trajetória com distância mínima é a seguinte: Luísa percorre  $17 - 4 = 13$  metros para ir da escola à lojinha,  $25 - 17 = 8$  metros para ir da lojinha ao supermercado, e  $25 - 4 = 21$  metros para ir do supermercado de volta para a escola. Assim, a resposta é  $13 + 8 + 21 = 42$  metros.

| Exemplo de entrada 3 | Exemplo de saída 3 |
|----------------------|--------------------|
| 1000<br>0<br>500     | 2000               |

## Gráfico de Barras

Nome do arquivo: `barras.c`, `barras.cpp`, `barras.pas`, `barras.java`, `barras.js` ou `barras.py`

Caique é o mais novo estagiário da OBI (*Organização de Brinquedos Infantis*) e está encarregado de preparar os slides que serão apresentados na próxima reunião de vendas do departamento. A fim de avaliar os futuros investimentos da empresa, o chefe do garoto pediu que ele fizesse um gráfico sobre a popularidade de diversos brinquedos.

Para conseguir esses dados, a OBI realizou previamente uma pesquisa com várias crianças, na qual cada uma escolheu seu brinquedo favorito dentre  $N$  opções disponíveis. Os resultados da pesquisa foram organizados em uma lista de  $N$  valores, onde o  $i$ -ésimo desses valores,  $X_i$ , indica quantos participantes da pesquisa preferem o  $i$ -ésimo brinquedo. Caique já possui essa lista e deve criar um gráfico de barras com base nessas informações.

Ele deve seguir a seguinte formatação padrão da empresa:

- O gráfico deve conter  $N$  colunas de mesma altura  $H$  compostas de números 0 e 1, representando os valores na ordem da lista.
- Não deve existir nenhuma linha completamente preenchida por números 0, ou seja,  $H$  deve ser igual ao maior valor  $X_i$  da lista.
- A base da  $i$ -ésima coluna deve ser composta de  $X_i$  números iguais a 1, enquanto os outros  $H - X_i$  números do topo devem ser iguais a 0.

Por exemplo, suponha que  $N = 4$  e a lista seja 4, 2, 5, 3. Nesse caso,  $H = 5$  e o gráfico de barras deve ser:

|   |   |   |   |
|---|---|---|---|
| 0 | 0 | 1 | 0 |
| 1 | 0 | 1 | 0 |
| 1 | 0 | 1 | 1 |
| 1 | 1 | 1 | 1 |
| 1 | 1 | 1 | 1 |

Caique ainda está muito ocupado com outros pedidos de seu chefe e pediu sua ajuda. Portanto, dada a lista de popularidade dos  $N$  brinquedos, ajude-o a construir um gráfico que siga as especificações da OBI.

### Entrada

A primeira linha da entrada contém um único inteiro  $N$ , a quantidade de brinquedos da pesquisa realizada pela OBI. A segunda linha da entrada contém  $N$  inteiros, o  $i$ -ésimo desses valores,  $X_i$ , indica a quantidade de participantes da pesquisa que preferem o  $i$ -ésimo brinquedo.

### Saída

A saída deve conter o gráfico de barras conforme especificado pela OBI, portanto, deve ser composta por  $H$  linhas, onde cada uma delas deve ser composta por  $N$  números, cada um sendo 0 ou 1, formando o gráfico desejado pela OBI.

## Restrições

É garantido que todo caso de teste satisfaz as restrições abaixo.

- $1 \leq N \leq 100$
- $0 \leq X_i \leq 100$ , para todo  $1 \leq i \leq N$
- Existe algum elemento não-nulo na lista, ou seja,  $\max(X_i) > 0$

## Informações sobre a pontuação

A tarefa vale 100 pontos. Estes pontos estão distribuídos em subtarefas, cada uma com suas **restrições adicionais** às definidas acima.

- **Subtarefa 1 (0 pontos):** Esta subtarefa é composta apenas pelos exemplos mostrados abaixo. Ela não vale pontos, serve apenas para que você verifique se o seu programa imprime o resultado correto para os exemplos.
- **Subtarefa 2 (30 pontos):**  $X_i \leq 2$ , para todo  $1 \leq i \leq N$ .
- **Subtarefa 3 (70 pontos):** Sem restrições adicionais.

## Exemplos

| Exemplo de entrada 1 | Exemplo de saída 1 |
|----------------------|--------------------|
| 4                    | 0 0 1 0            |
| 4 2 5 3              | 1 0 1 0            |
|                      | 1 0 1 1            |
|                      | 1 1 1 1            |
|                      | 1 1 1 1            |

*Explicação do exemplo 1:* Este exemplo foi mostrado no enunciado.

| Exemplo de entrada 2 | Exemplo de saída 2 |
|----------------------|--------------------|
| 5                    | 0 1 0 1 0          |
| 1 2 1 2 1            | 1 1 1 1 1          |

*Explicação do exemplo 2:* Perceba que este exemplo satisfaz às restrições da subtarefa 2.

| Exemplo de entrada 3 | Exemplo de saída 3 |
|----------------------|--------------------|
| 8                    | 0 0 0 0 0 0 0 1    |
| 1 2 3 4 5 6 7 8      | 0 0 0 0 0 0 1 1    |
|                      | 0 0 0 0 0 1 1 1    |
|                      | 0 0 0 0 1 1 1 1    |
|                      | 0 0 0 1 1 1 1 1    |
|                      | 0 0 1 1 1 1 1 1    |
|                      | 0 1 1 1 1 1 1 1    |
|                      | 1 1 1 1 1 1 1 1    |